

Exercice : La structure Pose et l'application à un point

Consigne

Écrire la structure `Pose`, avec une position et un quaternion, et la fonction qui applique une pose à un point : rotation puis translation. Le quaternion est donné par ses quatre composantes, déjà normalisé. Le programme lit une pose et un point, et affiche le point transformé.

Rappel

Une pose localise quelque chose : où il est, et comment il est tourné. Une position en mètres, une orientation en quaternion unitaire. C'est tout. Une pose n'a pas d'échelle.

L'ordre d'application est fixé : on tourne d'abord, on déplace ensuite.

$$p_{\text{espace}} = \text{orientation} \times p_{\text{entite}} + \text{position}$$

Structure Pose

```
#include <array>

struct Vec3
{
    float x, y, z;
};

struct Quat
{
    float w, x, y, z; // deja normalise
};

struct Pose
{
    Vec3 position; // en metres
    Quat orientation; // quaternion unitaire
};
```

Fonction d'application d'une pose à un point

```
// Rotation d'un vecteur par un quaternion unitaire :
// v' = v + 2 * cross(q.xyz, cross(q.xyz, v) + q.w * v)
Vec3 Rotation(const Quat& q, const Vec3& v)
{
    Vec3 u = { q.x, q.y, q.z };

    // cross(u, v)
    Vec3 uv = {
        u.y * v.z - u.z * v.y,
        u.z * v.x - u.x * v.z,
        u.x * v.y - u.y * v.x
    };

    // cross(u, uv) + q.w * v
    Vec3 t = {
        u.y * uv.z - u.z * uv.y + q.w * v.x,
        u.z * uv.x - u.x * uv.z + q.w * v.y,
        u.x * uv.y - u.y * uv.x + q.w * v.z
    };
}
```

```

};

// v + 2 * t
return {
    v.x + 2.0f * t.x,
    v.y + 2.0f * t.y,
    v.z + 2.0f * t.z
};
}

// Application d'une pose a un point : rotation puis translation.
Vec3 Appliquer(const Pose& pose, const Vec3& point)
{
    Vec3 tourne = Rotation(pose.orientation, point);
    return {
        tourne.x + pose.position.x,
        tourne.y + pose.position.y,
        tourne.z + pose.position.z
    };
}

```

Programme complet

```

#include <array>
#include <cstdio>

struct Vec3 { float x, y, z; };
struct Quat { float w, x, y, z; };
struct Pose { Vec3 position; Quat orientation; };

Vec3 Rotation(const Quat& q, const Vec3& v)
{
    Vec3 u = { q.x, q.y, q.z };
    Vec3 uv = {
        u.y * v.z - u.z * v.y,
        u.z * v.x - u.x * v.z,
        u.x * v.y - u.y * v.x
    };
    Vec3 t = {
        u.y * uv.z - u.z * uv.y + q.w * v.x,
        u.z * uv.x - u.x * uv.z + q.w * v.y,
        u.x * uv.y - u.y * uv.x + q.w * v.z
    };
    return {
        v.x + 2.0f * t.x,
        v.y + 2.0f * t.y,
        v.z + 2.0f * t.z
    };
}

Vec3 Appliquer(const Pose& pose, const Vec3& point)
{
    Vec3 tourne = Rotation(pose.orientation, point);
    return {
        tourne.x + pose.position.x,
        tourne.y + pose.position.y,
        tourne.z + pose.position.z
    };
}

```

```

}

int main()
{
    Pose pose;
    Vec3 point;

    // Lecture : position (3), quaternion (4), point (3)
    std::scanf("%f %f %f", &pose.position.x,
                  &pose.position.y,
                  &pose.position.z);
    std::scanf("%f %f %f %f", &pose.orientation.w,
                  &pose.orientation.x,
                  &pose.orientation.y,
                  &pose.orientation.z);
    std::scanf("%f %f %f", &point.x, &point.y, &point.z);

    Vec3 resultat = Appliquer(pose, point);

    std::printf("%.4f %.4f %.4f\n",
                resultat.x, resultat.y, resultat.z);

    return 0;
}

```

Exemple d'exécution

Pour l'entrée :

```

1.0 2.0 3.0
1.0 0.0 0.0 0.0
1.0 1.0 1.0

```

Le quaternion est l'identité ($w = 1, x = y = z = 0$). La rotation ne change rien. Le résultat est :

```

2.0000 3.0000 4.0000

```

Explication

- La rotation est appliquée d'abord : $v' = q v q^{-1}$, calculée ici avec la formule optimisée $v + 2 \cdot \text{cross}(u, \text{cross}(u, v) + w \cdot v)$.
- La translation est appliquée ensuite : on ajoute la position de la pose.
- L'ordre inverse ferait tourner l'objet autour de l'origine du monde au lieu de le faire tourner sur lui-même. Le mot pour cela est le **pivot**.

À retenir

L'ordre est fixé : rotation puis translation. Une pose n'a pas d'échelle. Si vous avez déjà vu un objet « partir en orbite » au lieu de tourner sur place, c'était un ordre inversé.